

FINAL PROJECT · ARTIFICIAL INTELLIGENCE

What the market *will ask next*

Forecasting food commodity prices in Cambodia — from World Food Programme price monitoring to a web app and a Telegram bot.

46 COMMODITIES · 76 MARKETS · XGBOOST, PORTED TO TYPESCRIPT

An early warning *only works if it arrives*

PROBLEM

No simple food-price forecast exists.

- Prices are public, but hard to use.
- People need warning before prices rise.
- A CSV is not enough for everyday users.



SOLUTION

Make the forecast easy to reach.

- 46 commodities and 76 markets.
- Choose a food and market.
- See price history, trend and forecast.

THE DATA

Twenty-three years *of market visits*

SOURCE	WFP Global Food Prices — Cambodia (HDX)
OBSERVATIONS	85,107
RANGE	2003-01-15 to 2026-03-15
COMMODITIES	50 raw
MARKETS	86 raw
TARGET	usdprice — price per unit in USD

FILTERING

85,107 rows

everything WFP has recorded



from 2019 onward

older reporting is sparse and pre-dollarisation



commodities with 300+ rows

a series needs history to have a lag



67,654 modelled rows

46 commodities · 76 markets · 3,280 series

The split is the *whole experiment*

Each (commodity, market, pricetype) forms a time series.
The model predicts the next observation in that series
from its own recent history.

THE TRAP

A random train/test split lets the model learn from
June to predict April. Prices are autocorrelated, so it
scores brilliantly and forecasts nothing. The split has
to be chronological.

CHRONOLOGICAL HOLD-OUT

Cut point	2025-04-15 (85th percentile of dates)
Train	57,878 rows · everything before the cut
Test	9,776 rows · everything after
Never seen	the test period, by any model, at any point

Ten features, built *inside each series*

PRICE HISTORY

lag_1

the previous reported price

lag_3

the price three reports back

rolling_mean_3

mean of the last three

CALENDAR

month

1-12, seasonality of harvest

quarter

1-4, coarser seasonality

IDENTITY

commodity_enc

which food

market_enc

which market

admin1_enc

which province

category_enc

cereals, meat, pulses ...

pricetype_enc

retail or wholesale

Lags are computed within each series (grouped shift), never across them — a rice price in Battambang can never leak into a pork price in Phnom Penh. Rows without three prior observations are dropped rather than imputed.

Beating “tomorrow *equals today*”

The naive baseline — predict the last price — already scores R^2 0.92, because prices are sticky. It is the number any model has to beat to be worth deploying.

MODEL	MAE (USD)	RMSE (USD)	R^2
Naive baseline (lag_1)	0.1799	0.4088	0.9239
Random Forest (retrained)	0.1690	0.3352	0.9489
XGBoost (final model)	0.1641	0.3303	0.9504

Random Forest reproduced locally from the notebook's settings (n=300, depth 15, seed 42).
XGBoost figures come from the trained price_model.pkl itself, evaluated on the held-out period.

RESULT

The model earns *its place*

\$0.164

MEAN ABSOLUTE ERROR

on held-out 2025-26 prices

0.950

R² ON THE TEST PERIOD

variance explained

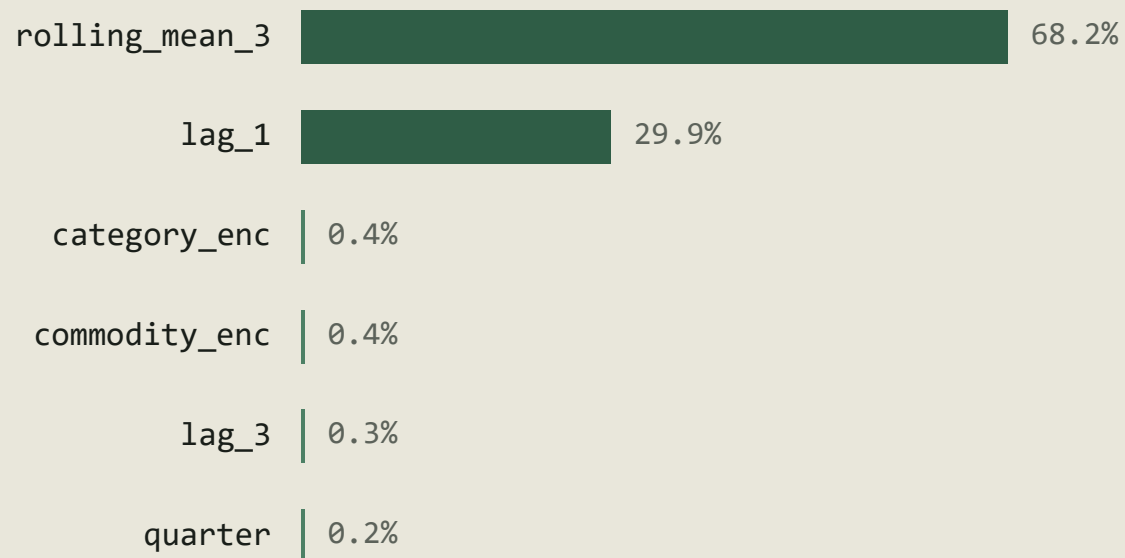
8.8%

BETTER THAN NAIVE

MAE reduction vs. last price

An average miss of about sixteen US cents on a basket whose prices mostly sit between \$0.30 and \$3.00 — accurate enough to plan around, not precise enough to trade on. Both halves of that sentence matter.

It learned to be *a very good smoother*



THE HONEST READING

98% of the model

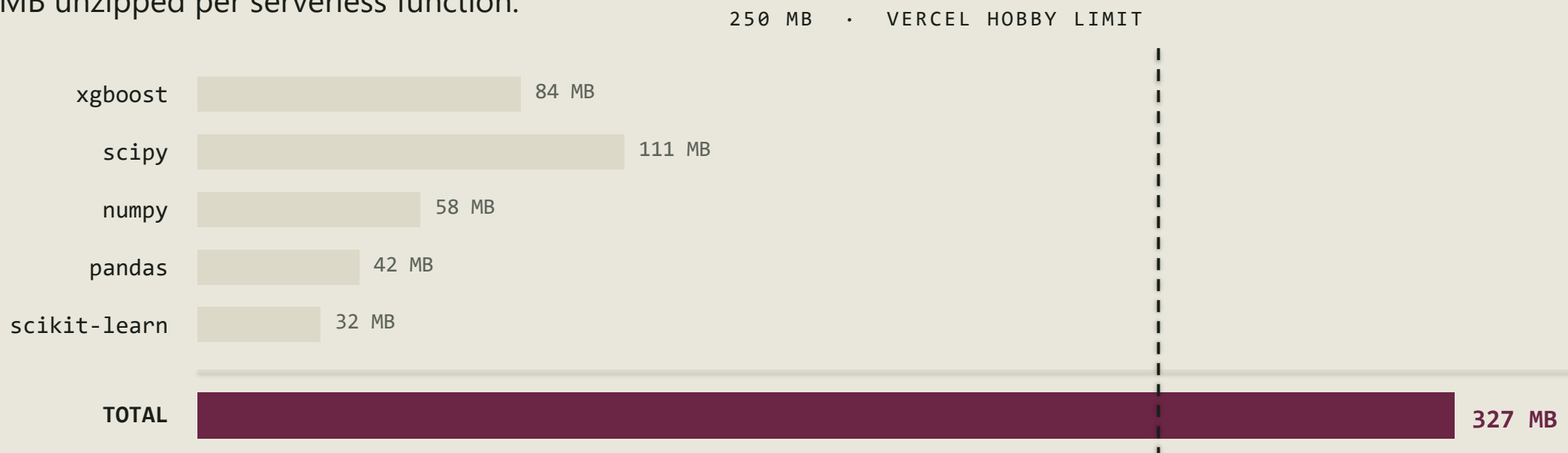
rests on two features: the recent mean and the last price.

Commodity, market, province and season together account for barely two percent. This is a well-calibrated smoother, not a market oracle — and saying so is part of the result.

The model fit.

The libraries didn't.

A notebook that only runs in Colab isn't a product. Deploying to Vercel's free tier meant meeting a hard limit: 250 MB unzipped per serverless function.



The deploy fails before a single prediction is ever served.

A boosted tree is *just a lot of if-statements*

Training needs XGBoost. Predicting does not. Inference is: walk each tree comparing one feature to one threshold, sum the leaf weights, add the base score. Fifteen lines in any language.

400

trees traversed

10

features each

2.4 MB

of JSON

0

ML dependencies

The model exports to JSON: tree arrays, encoder maps, feature order, and a 24-point price history per series. The deployed function ships none of the Python stack.

```
function rawPredict(features: number[]): number {  
  let total = BASE_SCORE  
  for (const tree of TREES) {  
    let node = 0  
    while (tree.l[node] !== -1) {  
      node = features[tree.f[node]] < tree.t[node]  
        ? tree.l[node]  
        : tree.r[node]  
    }  
    total += tree.w[node]  
  }  
  return total  
}
```

Two languages, *one number*

A port is only worth anything if it agrees with the original. The test suite pins the TypeScript traversal to the Python model's own answer for a known series — a wrong feature order or a flipped comparison changes that number, and nothing else would catch it.

19 tests cover the traversal, the fuzzy matching, the commodity/market splitter and every failure path.

REFERENCE CASE

commodity	Rice (mixed, low quality)
market	Phnom Penh · Wholesale
last recorded	2022-06 · \$0.44

python	xgboost	0.458068
--------	---------	-----------------

typescript	port	0.458068
------------	------	-----------------

agree to 1e-5 · asserted on every test run

A forecast people *can actually open*

The model is not left in the notebook. Nuxt serves the same prediction engine through a web page, API routes and a Telegram webhook.

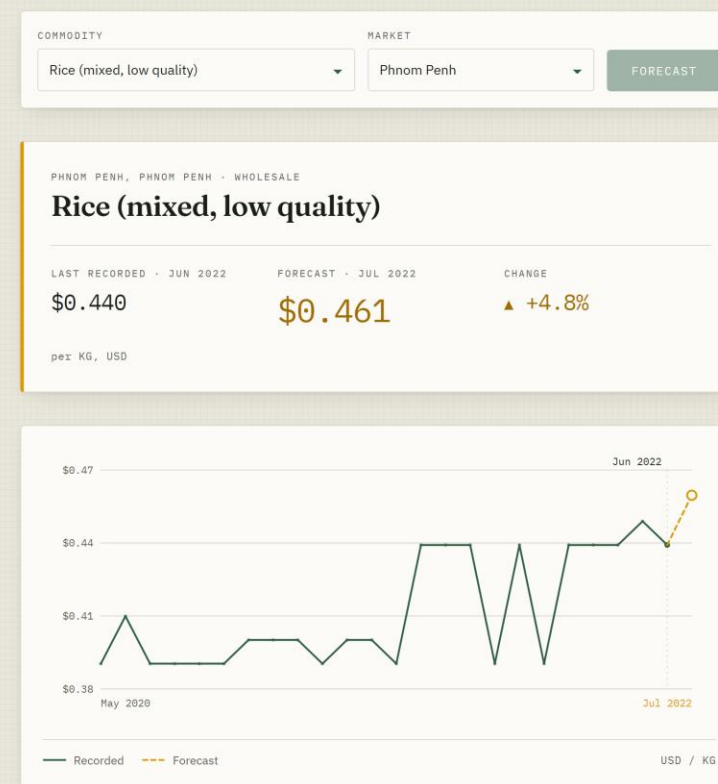
- Browser** Choose a commodity and market, then see the latest price, forecast and chart.
- API** `/api/predict` returns the same forecast for the web UI and outside clients.
- Telegram** `/predict rice phnom penh` gives a fast text answer with a small sparkline.
- Deployment** Vercel only receives the Nuxt app and JSON artifacts, not Python ML libraries.

One trained model, one TypeScript engine, two user-facing surfaces.

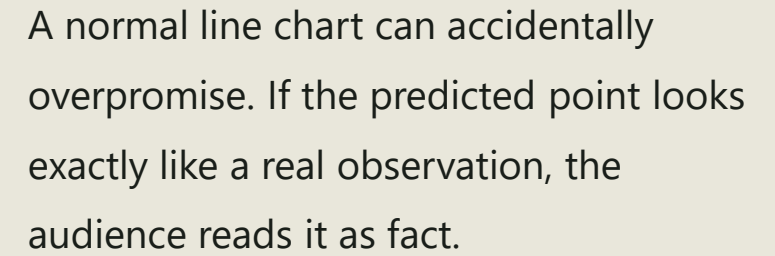
WFP PRICE MONITORING · CAMBODIA

What the market *will ask next*

Next-period price forecasts for **46** commodities across **76** markets, from a gradient-boosted model trained on World Food Programme observations.



Forecasts are model estimates, not official prices. Each one projects a single reporting period past the last genuine observation for that commodity and market.



- The visual design protects the user from confusing an estimate with official market data.

LIMITS

Strong result, *clear next step*

The model performs well, but the honest boundary is important: it forecasts from recent prices, so shocks and stale series remain hard.

WHAT WORKS

$R^2 = 0.950$; average error about \$0.164 on held-out prices.

WHAT IS MISSING

Fuel cost, rainfall, import pressure, holidays and local shocks are not in the model.

WHAT I WOULD ADD

Prediction intervals, external economic/weather features and scheduled monthly retraining.

This is a credible forecasting prototype, not a replacement for field data or official price monitoring.

FINAL TAKEAWAY

Public data becomes useful *when it answers tomorrow*

Cambodia already has valuable food-price records. This project shows how AI can turn those records into a small, explainable forecasting tool that ordinary users can reach from a browser or chat message.

DATA	WFP Cambodia food prices
MODEL	XGBoost regression, tested chronologically
PRODUCT	Nuxt app, API and Telegram bot